



FUNDAÇÃO UNIVERSIDADE FEDERAL DO VALE DO SÃO FRANCISCO

COLEGIADO DE ENGENHARIA DA COMPUTAÇÃO

Av. Antônio Carlos Magalhães, 510, Country Club, Juazeiro-BA - CEP: 48.902-300

Fone/Fax: (74) 2102-7636, cecomp.univasf.edu.br

Sistemas Distribuídos I

Prof. Jairson B. Rodrigues

Atividade Prática I – gRPC e Protocol Buffers

Tarefa: criar um sistema de gerenciamento de tarefas **simples**, com um serviço central (servidor gRPC) e uma aplicação cliente que o consome.

Objetivo: proporcionar experiência prática com gRPC¹ (*Google Remote Procedure Call*) e Protocol Buffers, potencializando que o aluno compreenda a comunicação eficiente e tipada entre serviços distribuídos.

Contexto

Considere que uma equipe precisa gerenciar tarefas de maneira centralizada. Para isso, será desenvolvido um sistema cliente-servidor em que:

- o **servidor** é responsável por armazenar, listar, criar e atualizar as tarefas²;
- o **cliente** envia solicitações ao servidor para executar essas operações;
- a **comunicação** entre cliente e servidor será realizada através de gRPC, usando Protocol Buffers para definir a estrutura dos dados (as tarefas) e a interface do serviço.

¹ <https://grpc.io/>

² O discente está livre para pensar a modelagem de uma tarefa como melhor entender. Alguns dados são básicos: id, título, descrição, status, data limite, responsável(eis).



FUNDAÇÃO UNIVERSIDADE FEDERAL DO VALE DO SÃO FRANCISCO

COLEGIADO DE ENGENHARIA DA COMPUTAÇÃO

Av. Antônio Carlos Magalhães, 510, Country Club, Juazeiro-BA - CEP: 48.902-300

Fone/Fax: (74) 2102-7636, cecomp.univasf.edu.br

Estrutura do projeto

- **Modelagem** criar um arquivo *.proto* com a definição do serviço e as mensagens de dados (por exemplo: o objeto *Tarefa* e as operações *ListarTarefas*, *CriarTarefa*, etc). O intuito reside em compreender como Protocol Buffers funcionam como uma Linguagem de Definição de Interface (IDL) para gRPC.
- **Servidor gRPC**: implementar a lógica do servidor que executa as operações definidas no arquivo *.proto*. Isso inclui a geração de código a partir do arquivo *.proto* para a linguagem escolhida³ e a codificação da lógica de negócios para gerenciar as tarefas em memória (por exemplo, em um *hash map* simples).
- **Cliente gRPC**: O aluno irá desenvolver uma aplicação cliente que utiliza o código gerado do arquivo *.proto* para fazer chamadas remotas ao servidor. O cliente deve ser capaz de invocar os métodos RPC do servidor para interagir com o sistema de tarefas.

Requisitos

Implementar as seguintes funcionalidades:

- **CriarTarefa**: o cliente envia os dados de uma nova tarefa (título, descrição etc) e o servidor a adiciona⁴, retornando a tarefa criada com um ID único. Sugestão: utilizar *Universally Unique Identifier (UUID)*.
- **ListarTarefas**: o cliente solicita todas as tarefas existentes e o servidor retorna a lista.
- **AtualizarTarefa**: O cliente envia uma tarefa com um ID existente e os novos dados. O servidor atualiza a tarefa correspondente.
- **DeletarTarefa**: O cliente envia o ID de uma tarefa e o servidor a remove.

³ Sugestão: Python [<https://grpc.io/docs/languages/python/>]

⁴ Para efeitos desta avaliação, considere armazenar cada tarefa em um arquivo separado em uma pasta específica do servidor. O foco do projeto está na troca de mensagens, na definição de interface e serviços remotos. Questões relacionadas com desempenho do armazenamento e recuperação serão minimizadas neste contexto.



FUNDAÇÃO UNIVERSIDADE FEDERAL DO VALE DO SÃO FRANCISCO

COLEGIADO DE ENGENHARIA DA COMPUTAÇÃO

Av. Antônio Carlos Magalhães, 510, Country Club, Juazeiro-BA - CEP: 48.902-300

Fone/Fax: (74) 2102-7636, cecomp.univasf.edu.br

Linguagem de programação: gRPC suporta uma variedade de linguagens. Inclusive, cliente e servidor podem ter linguagens diferentes. Sugestão do professor: utilize Python.

Apresentação em sala (10 a 15 minutos / não precisa de slides)

- criação e compilação de um arquivo .proto para definir a interface de um serviço.
- trechos da implementação do servidor e do cliente gRPC e demonstração do funcionamento
- uma explanação sobre a importância de protocol buffers para serialização de objetos distribuídos e uma comparação com JSON

Obs 1: cliente e servidor podem ser processos em linha de comando

Obs 2: realizar demonstração com dois clientes e um servidor em máquinas separadas com IPs distintos. Considere utilizar virtuabox + vagrant ou docker.